

GO (GOLANG) COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to Go (Golang)

- 2. Variables and Data Types
- 3. Conditional Statements (if, else if, else)

★ 4. Loops - the 'for' Loop

★ 5. Functions

★ 6. Multiple Return Values

★ 7. Arrays and Slices

- 8. Maps

★ 9. Structs

- 10. Pointers

★ 11. Interfaces

★ 12. Goroutines

- 13. Channels
- 14. Error Handling

★ 1. Introduction to Go (Golang) [IMPORTANT TOPIC]

Explanation:

Go (also called Golang) is a programming language created by Google. It was designed to be simple, fast, and very good at handling many tasks at the same time, which makes it popular for building servers and cloud software.

Key Points:

- Simple syntax, easy to learn compared to many languages.
- Compiles into a single fast program, no separate installer needed to run it.
- Has built-in support for running many tasks at once (concurrency).

Example Code:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

Output:

Hello, World!

 *Note: Every Go program must belong to a 'package', and an executable program must have 'package main'.*

2. Variables and Data Types

Explanation:

A variable stores a value. Go can automatically detect a variable's type using ':=', or we can state the type directly using 'var'.

Example Code:

```
var age int = 21
name := "Pravendra" // Go detects this is a string

fmt.Println(name, age)
```

Output:

```
Pravendra 21
```

3. Conditional Statements (if, else if, else)

Explanation:

Conditional statements let the program make decisions. Note that Go does NOT use round brackets () around the condition, unlike many other languages.

Example Code:

```
marks := 75

if marks >= 90 {
    fmt.Println("Grade A")
} else if marks >= 60 {
    fmt.Println("Grade B")
} else {
    fmt.Println("Grade C")
}
```

Output:

Grade B

★ 4. Loops - the 'for' Loop *[IMPORTANT TOPIC]*

Explanation:

Go has only ONE type of loop: 'for'. But it is flexible enough to work like a normal for loop, a while loop, or an infinite loop, just by changing how it's written.

Example Code:

```
for i := 1; i <= 3; i++ {  
    fmt.Println("Count:", i)  
}
```

Output:

```
Count: 1  
Count: 2  
Count: 3
```

 *Note: Go has no separate 'while' keyword — a for loop with only a condition works exactly like a while loop.*

★ 5. Functions *[IMPORTANT TOPIC]*

Explanation:

A function is a reusable block of code. In Go, the data type is written AFTER the variable name, which is different from many other languages.

Example Code:

```
func addNumbers(a int, b int) int {  
    return a + b  
}  
  
func main() {  
    total := addNumbers(5, 7)  
    fmt.Println("Total:", total)  
}
```

Output:

Total: 12

★ 6. Multiple Return Values *[IMPORTANT TOPIC]*

Explanation:

A unique feature of Go is that a function can return MORE THAN ONE value at once. This is very commonly used to return both a result AND an error at the same time.

Example Code:

```
func divide(a int, b int) (int, string) {  
    if b == 0 {  
        return 0, "cannot divide by zero"  
    }  
    return a / b, ""  
}  
  
result, errMsg := divide(10, 2)  
fmt.Println(result, errMsg)
```

Output:

5

★ 7. Arrays and Slices [IMPORTANT TOPIC]

Explanation:

An array has a FIXED size in Go. A Slice is much more commonly used — it is like an array, but its size can grow or shrink as needed.

Example Code:

```
numbers := []int{10, 20, 30} // this is a slice
numbers = append(numbers, 40)

fmt.Println(numbers)
fmt.Println(numbers[0])
```

Output:

```
[10 20 30 40]
10
```

8. Maps

Explanation:

A map stores data as key-value pairs, similar to a dictionary, letting us quickly find a value using its key.

Example Code:

```
marks := map[string]int{
    "Pravendra": 90,
    "Aman":      85,
}

fmt.Println(marks["Pravendra"])
```

Output:

```
90
```

★ 9. Structs *[IMPORTANT TOPIC]*

Explanation:

A struct groups related data together under one name. Go does not have classes like other OOP languages — structs are Go's way of organizing related data.

Example Code:

```
type Student struct {  
    Name string  
    Age  int  
}  
  
s1 := Student{Name: "Pravendra", Age: 21}  
fmt.Println(s1.Name, s1.Age)
```

Output:

```
Pravendra 21
```

10. Pointers

Explanation:

A pointer stores the memory address of a variable, instead of the value itself. Go's pointers are simpler and safer to use than in a language like C.

Example Code:

```
age := 21
ptr := &age // ptr stores the address of age

fmt.Println(*ptr) // *ptr gives the value at that address
```

Output:

```
21
```

★ 11. Interfaces *[IMPORTANT TOPIC]*

Explanation:

An interface defines a set of method names that a type must have, WITHOUT saying how they must work. Any struct that has those exact methods automatically 'satisfies' that interface — there is no need to write 'implements' like in other languages.

Example Code:

```
type Shape interface {
    Area() float64
}

type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return 3.14 * c.Radius * c.Radius
}

var s Shape = Circle{Radius: 2}
fmt.Println(s.Area())
```

Output:

12.56

★ 12. Goroutines [IMPORTANT TOPIC]

Explanation:

A goroutine is a lightweight task that Go can run AT THE SAME TIME as other code, using very little memory. This is Go's famous feature for handling many tasks together, like handling thousands of website visitors at once.

Example Code:

```
func sayHello() {  
    fmt.Println("Hello from goroutine!")  
}  
  
func main() {  
    go sayHello()    // runs at the same time, in the background  
    fmt.Println("Main function continues...")  
}
```

Output:

```
Main function continues...  
Hello from goroutine! (order may vary)
```

 *Note: Just adding the word 'go' before a function call runs it as a goroutine — this simplicity is why Go is so popular for concurrent programs.*

13. Channels

Explanation:

A channel is used to safely SEND data between different goroutines, allowing them to communicate and work together without conflicts.

Example Code:

```
func main() {  
    messages := make(chan string)  
  
    go func() {  
        messages <- "Hello from goroutine"  
    }()  
  
    msg := <-messages  
    fmt.Println(msg)  
}
```

Output:

```
Hello from goroutine
```

14. Error Handling

Explanation:

Go does NOT use try-catch like many other languages. Instead, functions simply RETURN an error value, and the calling code checks if it is not empty (nil).

Example Code:

```
import "errors"

func divide(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("cannot divide by zero")
    }
    return a / b, nil
}

result, err := divide(10, 0)
if err != nil {
    fmt.Println("Error:", err)
} else {
    fmt.Println(result)
}
```

Output:

Error: cannot divide by zero